TuEvento: Manual Técnico

Jhampier Santos Ortiz y Cristofer David Lozano Contreras

Servicio Nacional de Aprendizaje (SENA)

Análisis y Desarrollo de Software (ADSO), Ficha 3145556

Instructora: Karol Daniela Correa

9 de octubre de 2026

Tabla de contenido

Introducción **5**

Propósito . . . 5

Alcance . . . 5

Audiencia . . . 5

Documentos relacionados . . . 5

Arquitectura del sistema **6**

Visión general . . . 6

Flujos de comunicación . . . 7

Estructura por capas del backend . . . 8

Pila tecnológica **8**

Dependencias del backend . . . 10

Dependencias del frontend web, la aplicación móvil y los servicios auxiliares . . . 12

Repositorio y organización del código **14**

Estructura general . . . 14

Módulos del backend . . . 15

Organización del frontend . . . 16

Árbol de directorios . . . 16

Backend **17**

Perfiles de configuración . . . 17

Servicios de Docker Compose . . . 17

Variables de entorno . . . 18

Autenticación y autorización . . . 21

Autenticación de WebSocket . . . 22

Manejo de errores . . . 22

API REST y servicios **22**

Módulo security 23
Módulo event 25
Módulos payment, ticket y wallet 27
Otros controladores 28
Base de datos **28**
Motor y acceso 28
Migraciones del esquema 29
Resumen del modelo de datos 29
Tablas por dominio 31
Frontend web **34**
Aplicación móvil **35**
Integraciones **35**
Inicio de sesión con Google y Facebook 35
Notificaciones por correo 35
Almacenamiento de objetos 35
Redis 35
Cascada de moderación de imágenes 36
Pasarela de pago 36
Patrones de diseño y decisiones técnicas **37**
Patrones arquitectónicos 37
Patrones de diseño 37
Decisiones técnicas 37
Seguridad **37**
Controles implementados 37
Endurecimiento pendiente 38

Pruebas y calidad **39**

Pruebas existentes 39

Integración continua 39

Manifiestos de Kubernetes 40

Mantenimiento y extensión **41**

Convenciones 41

Cómo agregar un módulo al backend 41

Solución de problemas 41

Limitaciones conocidas **42**

Glosario **42**

Referencias **43**

Introducción

Propósito

Este manual documenta la arquitectura, la implementación y el funcionamiento interno de TuEvento, una plataforma de gestión de eventos desarrollada por el equipo CapySoft. Está escrito para desarrolladores, arquitectos de software y personal técnico que mantienen, extienden u operan el sistema. Describe el sistema tal como existe en el repositorio del proyecto (Santos Ortiz y Lozano Contreras, 2026b).

Alcance

El documento cubre la arquitectura del sistema, la pila tecnológica y sus versiones, la organización del código, la configuración, la capa de datos, las integraciones, las decisiones de diseño, la seguridad, las prácticas de pruebas y calidad, y las pautas de mantenimiento y extensión. Los pasos de instalación y despliegue no se repiten aquí; se describen en el manual de instalación (Santos Ortiz y Lozano Contreras, 2026a).

Audiencia

Los desarrolladores backend y frontend usan este manual para comprender la estructura del código. El personal de DevOps lo usa para entender la configuración y los servicios. Los arquitectos lo usan para evaluar las decisiones técnicas y los evaluadores lo usan para conocer la funcionalidad que debe verificarse.

Documentos relacionados

Este manual complementa el manual de instalación, la Especificación de Requisitos de Software (SRS), los documentos de análisis y diseño y la propuesta técnica del proyecto.

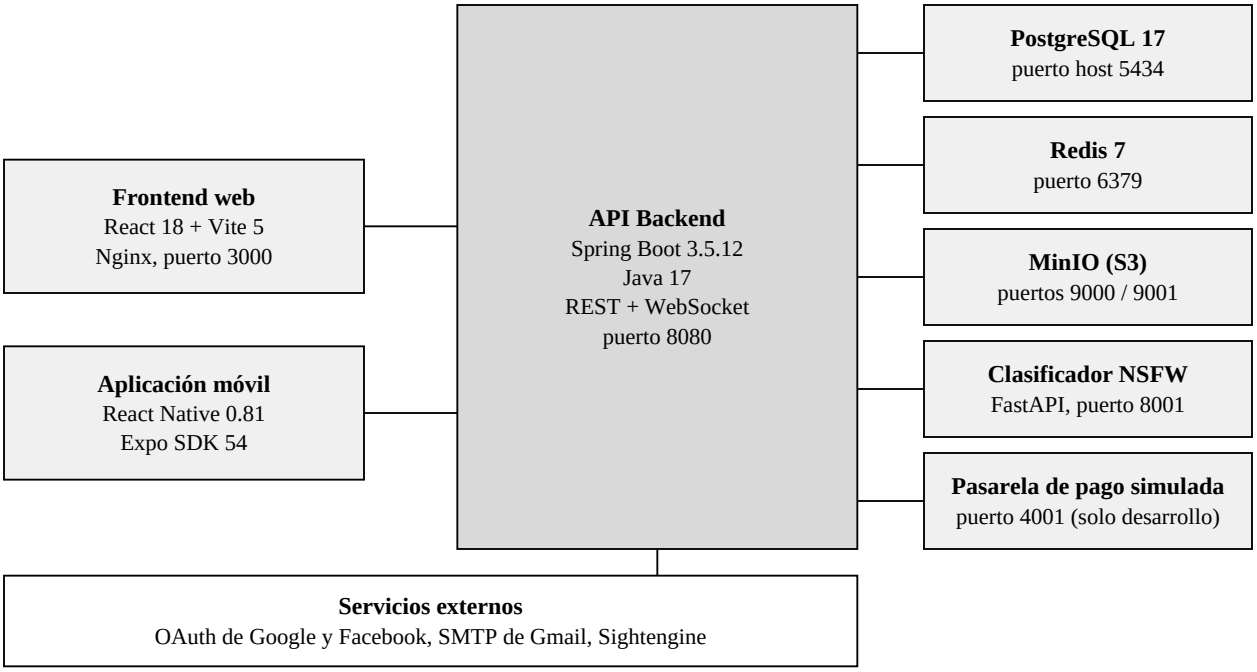
Arquitectura del sistema

Visión general

TuEvento se compone de tres capas desacopladas (un frontend web, un frontend móvil y una API REST backend) más la infraestructura de apoyo. Los servicios de la plataforma se orquestan con Docker Compose mediante el archivo `docker-compose.yml` de la raíz del repositorio (Docker, Inc., s. f.). La Figura 1 muestra los componentes y cómo se conectan, y la Tabla 1 lista cada componente con su tecnología y su puerto publicado en el host.

Figura 1

Arquitectura del sistema de TuEvento



Nota. Elaboración propia a partir del docker-compose.yml y del código fuente del proyecto. Las notificaciones en tiempo real usan WebSocket (STOMP) entre los clientes y el backend.

Tabla 1

Componentes de la plataforma y puertos del host

Componente	Tecnología	Puerto host
Frontend web	React 18 + Vite 5, servido por Nginx en el contenedor	3000
Frontend móvil	React Native 0.81 / Expo SDK 54	No aplica
API REST backend	Spring Boot 3.5.12, código fuente en Java 17; la imagen Docker ejecuta un JRE 21	8080
Base de datos	PostgreSQL 17	5434 (contenedor 5432)
Caché	Redis 7 (sin contraseña por defecto)	6379
Almacenamiento de objetos	MinIO (compatible con S3)	9000 (API), 9001 (consola)
Moderación de imágenes	Clasificador NSFW (Python, FastAPI, opennsfw2)	8001 (contenedor 8000)
Pasarela de pago (simulada)	Fake Payment Gateway, solo desarrollo	4001
Base de datos de la pasarela	Instancia PostgreSQL dedicada	5435
Integración continua	Jenkins	8082, 50000
Calidad de código	SonarQube	9002

Nota. Los valores se verificaron contra el repositorio y el manual de instalación. Jenkins y SonarQube no son necesarios para ejecutar la aplicación.

Flujos de comunicación

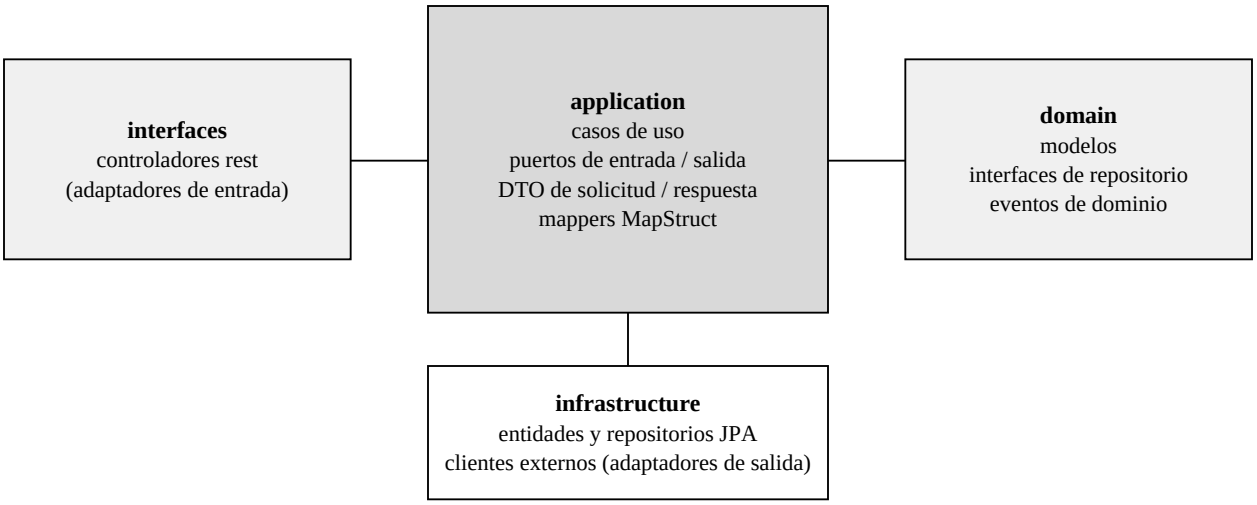
Los clientes se autentican con una solicitud de inicio de sesión al backend y reciben JSON Web Tokens (Jones et al., 2015). Las llamadas posteriores a la API llevan el token de acceso como token portador (bearer). Las notificaciones en tiempo real usan una conexión WebSocket al endpoint `/ws`, autenticada con el token. Las imágenes que suben los usuarios pasan por la cascada de moderación descrita en la sección Integraciones antes de almacenarse en MinIO. Los pagos se procesan a través de la pasarela de pago, que notifica al backend mediante un webhook protegido por un secreto compartido (`WEBHOOK_SECRET`).

Estructura por capas del backend

El backend sigue una arquitectura hexagonal (puertos y adaptadores) (Cockburn, 2005) con módulos orientados al dominio (Evans, 2003). Cada módulo separa su modelo de dominio, sus casos de uso de aplicación, sus adaptadores de infraestructura y sus interfaces de entrada, como se muestra en la Figura 2.

Figura 2

Estructura hexagonal de un módulo del backend



Nota. Elaboración propia a partir de la estructura de paquetes de com.capysoft.tuevento.modules.

Pila tecnológica

Las siguientes tablas resumen las principales tecnologías usadas en cada parte del sistema. Las versiones se tomaron de los archivos de dependencias del proyecto (Spring, s. f.-a; React, s. f.; Expo, s. f.).

Tabla 2

Tecnologías del backend

Área	Tecnología	Versión / detalle
Framework	Spring Boot	3.5.12
Lenguaje	Java	17 (código fuente); JRE 21 en la imagen Docker
Construcción	Maven	Maven wrapper (mvnw)
Persistencia	Spring Data JPA, PostgreSQL, HikariCP	PostgreSQL 17
Migraciones	Liquibase	Changelogs YAML numerados
Seguridad	Spring Security, jjwt, BCrypt	jjwt 0.12.6; costo BCrypt 12
Inicio de sesión social	OAuth 2.0 (Google, Facebook)	Cliente OAuth2 de Spring
Tiempo real	Spring WebSocket	STOMP
Correo	Spring Mail	SMTP de Gmail
Almacenamiento	AWS S3 SDK para MinIO	2.25.60
Mapeo	MapStruct	1.6.3
Documentación de la API	SpringDoc OpenAPI	2.8.8
Operación	Spring Boot Actuator	Salud y métricas
Productividad	Lombok, Bean Validation	No aplica

Nota. Spring Kafka está declarado como dependencia, pero no se implementó ningún productor ni consumidor (véase Limitaciones conocidas).

Tabla 3

Tecnologías del frontend web

Área	Tecnología	Versión
Framework	React	18.2.0
Herramienta de build	Vite	5.0.0
Enrutamiento	React Router DOM	6.20.0
Estilos	Tailwind CSS	4.2.2
Iconos	Lucide React	1.7.0
Editor de layouts	Konva y react-konva	9.3.22 y 18.2.10
Mapas	Leaflet y react-leaflet	1.9.4 y 4.2.1
Inicio de sesión social	@react-oauth/google	0.13.5
Tiempo real	@stomp/stompjs y sockjs-client	7.3.0 y 1.6.1
Fechas	date-fns	3.0.0

Nota. El editor de layouts usa Konva (Konva, s. f.) y los mapas usan Leaflet (Leaflet, s. f.).

Tabla 4

Tecnologías de la aplicación móvil

Área	Tecnología	Versión
Framework	React Native y Expo	0.81.5 y 54.0.33
Biblioteca	React	19.1.0
Navegación	React Navigation	7.x
Estilos	NativeWind	2.0.11
Gráficos	@shopify/react-native-skia	2.11.0
Autenticación	Expo Auth Session, jwt-decode	7.0.10 y 4.0.0
HTTP y formularios	Axios, React Hook Form	1.15.0 y 7.72.1

Nota. La aplicación móvil no forma parte del despliegue con Docker (React Native, s. f.).

Los servicios auxiliares incluyen el clasificador NSFW, construido con FastAPI (FastAPI, s. f.) y el modelo opennsfw2 (bhky, s. f.), y la Fake Payment Gateway, que tiene su propia base de datos PostgreSQL. La orquestación usa Docker Compose, la integración continua usa Jenkins (Jenkins, s. f.), el análisis de código usa SonarQube (SonarSource, s. f.) y se proporcionan manifiestos de Kubernetes para el despliegue en clúster (The Kubernetes Authors, s. f.).

Dependencias del backend

La Tabla 5 lista las dependencias declaradas en el `pom.xml` del backend, agrupadas por función.

Tabla 5

Dependencias del backend (pom.xml)

Grupo	Artefacto	Versión	Propósito
Web y framework	spring-boot-starter-parent	3.5.12	Framework base
	spring-boot-starter-web	3.5.12	API REST y controladores
	spring-boot-starter-actuator	3.5.12	Monitoreo y métricas
	spring-boot-starter-websocket	3.5.12	WebSocket y STOMP
Seguridad	spring-boot-starter-security	3.5.12	Autenticación y autorización
	spring-boot-starter-oauth2-client	3.5.12	OAuth 2.0 (Google, Facebook)
	spring-boot-starter-oauth2-resource-server	3.5.12	Validación de JWT
	jjwt-api, jjwt-impl, jjwt-jackson	0.12.6	Generación y validación de JWT
Persistencia	spring-boot-starter-data-jpa	3.5.12	ORM y repositorios JPA
	spring-boot-starter-data-redis	3.5.12	Caché y Redis
	postgresql	runtime	Controlador de PostgreSQL
	liquibase-core	—	Migraciones de base de datos
	spring-session-data-redis	—	Sesiones distribuidas en Redis
Mensajería	spring-kafka	—	Apache Kafka (sin uso en el código)
	spring-boot-starter-mail	3.5.12	Envío de correo SMTP
Utilidades	lombok	—	Reducción de código repetitivo
	mapstruct, mapstruct-processor	1.6.3	Mapeo entre DTO y entidades
	spring-dotenv	4.0.0	Carga de variables desde .env
	software.amazon.awssdk:s3	2.25.60	Cliente S3 para MinIO
	spring-boot-starter-validation	3.5.12	Validaciones de entrada
Documentación	springdoc-openapi-starter-webmvc-ui	2.8.8	OpenAPI y Swagger UI
Monitoreo	micrometer-tracing-bridge-brave	—	Trazabilidad
Pruebas	spring-boot-starter-test, spring-security-test, spring-kafka-test	test	Frameworks de prueba
	h2	test	Base de datos en memoria para pruebas
	jacoco-maven-plugin	0.8.12	Cobertura de código
	sonar-maven-plugin	3.9.1.2184	Análisis con SonarQube

Nota. Fuente: pom.xml del backend. Un guion indica que el pom.xml no declara una versión explícita; en esos casos la versión la gestiona Spring Boot. El groupId de cada artefacto se consulta en el pom.xml.

Dependencias del frontend web, la aplicación móvil y los servicios auxiliares

Tabla 6

Dependencias del frontend web (package.json)

Tipo	Paquete	Versión	Propósito
dependencies	react, react-dom	^18.2.0	Biblioteca de interfaz
	react-router-dom	^6.20.0	Enrutamiento
	@react-oauth/google	0.13.5	Inicio de sesión con Google
	@stomp/stompjs, sockjs-client	^7.3.0, ^1.6.1	Cliente WebSocket
	konva, react-konva	^9.3.22, ^18.2.10	Editor de layouts
	leaflet, react-leaflet	^1.9.4, ^4.2.1	Mapas
	date-fns	^3.0.0	Manejo de fechas
	lucide-react	^1.7.0	Iconos
devDependencies	vite, @vitejs/plugin-react	^5.0.0, ^4.2.0	Servidor y build
	tailwindcss, @tailwindcss/postcss	^4.2.2	Estilos
	autoprefixer, postcss	^10.4.27, ^8.5.8	Procesamiento de CSS

Nota. Fuente: package.json de TuEventoFrontend/web.

Tabla 7

Dependencias de la aplicación móvil (package.json)

Tipo	Paquete	Versión	Propósito
dependencies	expo	~54.0.33	Plataforma de desarrollo
	react, react-native	19.1.0, 0.81.5	Interfaz nativa
	@react-navigation/native, native-stack, bottom-tabs	^7.2.2, ^7.14.10, ^7.15.9	Navegación
	expo-auth-session	~7.0.10	OAuth móvil
	@react-native-async-storage/async-storage	2.2.0	Almacenamiento local
	@stomp/stompjs, sockjs-client	^7.3.0, ^1.6.1	Cliente WebSocket
	axios	^1.15.0	Cliente HTTP
	expo-image-picker, expo-document-picker	~17.0.11, ~14.0.8	Selección de imágenes y documentos
	expo-linear-gradient	~15.0.8	Degradados
	react-hook-form, jwt-decode	^7.72.1, ^4.0.0	Formularios y decodificación de JWT
	nativewind, @shopify/react-native-skia	^2.0.11, ^2.11.0	Estilos y gráficos 2D
devDependencies	typescript, @types/react, babel-preset-expo, tailwindcss	~5.9.2, ~19.1.10, ~54.0.10, ^3.3.2	Herramientas de desarrollo

Nota. Fuente: package.json de TuEventoFrontend/movil.

Tabla 8

Dependencias de los servicios auxiliares

Servicio	Dependencia	Versión	Propósito
NSFW (Python)	fastapi	0.111.0	API web
	uvicorn	0.30.1	Servidor ASGI
	pillow	10.3.0	Procesamiento de imágenes
	opennsfw2	0.13.3	Clasificador NSFW
	tensorflow	2.16.2	Motor de aprendizaje automático
Fake Payment Gateway	spring-boot-starter-parent, -web, -data-jpa, -validation	3.5.0	Servicio REST con persistencia
	postgresql, lombok	runtime, —	Controlador y reducción de código

Nota. Fuente: requirements.txt de nsfw-service y pom.xml de fake-payment-gateway.

Repositorio y organización del código

Estructura general

El código fuente del proyecto está alojado en un repositorio público de GitHub (Santos Ortiz y Lozano Contreras, 2026b). La Tabla 9 describe su organización de nivel superior.

Tabla 9

Estructura de nivel superior del repositorio

Ruta	Contenido
`TuEventoBackend/tu-evento/`	Backend Spring Boot
`TuEventoFrontend/web/`	Frontend web en React
`TuEventoFrontend/movil/`	Aplicación móvil React Native (Expo)
`nsfw-service/`	Clasificador de imágenes en Python
`fake-payment-gateway/`	Pasarela de pago simulada
`k8s/`	Manifiestos de Kubernetes (8 archivos YAML)
`jenkins/` y `Jenkinsfile`	Configuración de integración continua
`docker-compose.yml`	Orquestación de servicios
`.env.example`	Plantilla de variables de entorno

Módulos del backend

El paquete base del backend es `com.capysoft.tuevento`. El paquete `shared` contiene el código transversal (configuración global, seguridad, configuración de WebSocket y el manejador global de excepciones), y el paquete `modules` contiene los módulos de dominio listados en la Tabla 10. Dentro de cada módulo, los paquetes `domain`, `application`, `infrastructure` e `interfaces` mantienen las capas separadas.

Tabla 10

Módulos de dominio del backend

Módulo	Responsabilidad
security	Autenticación, usuarios, roles, activación de cuenta y recuperación de contraseña
profile	Perfiles de usuario
event	Ciclo de vida de eventos, calificaciones y comentarios, medios y layouts
category	Categorías de eventos
geolocation	Departamentos, ciudades y sitios
section	Secciones de eventos
seat	Asientos y reservas
ticket	Tickets, órdenes y check-in
payment	Procesamiento de pagos e integración con la pasarela
wallet	Billetera digital
notification	Notificaciones por correo y en tiempo real
storage	Almacenamiento de archivos y manejo de imágenes
theme	Temas de la interfaz
language	Internacionalización

Nota. Los nombres de los módulos corresponden a los directorios bajo src/main/java/com/capysoft/tuevento/modules.

Organización del frontend

El frontend web mantiene su código en `src/` con los directorios `assets`, `components`, `constants`, `context`, `data`, `hooks`, `layouts`, `pages`, `services`, `styles`, `theme` y `utils`. El estado global lo proveen proveedores de contexto de React para autenticación, tema y notificaciones. El directorio `services` contiene los clientes de la API, con `httpClient.js` como cliente HTTP compartido. La aplicación móvil sigue una organización similar, con `navigation/AppNavigator.jsx` para la navegación, `screens` para las pantallas y `services/api.js` como cliente base de la API.

Árbol de directorios

Los siguientes árboles muestran la estructura real del backend, del frontend web, de la aplicación móvil y de los servicios auxiliares.

Backend (src/main/java/com/capysoft/tuevento):

```
TuEventoApplication.java
modules/
  category/  event/  geolocation/  language/  notification/
  payment/   profile/  seat/  section/  security/
  storage/   theme/  ticket/  wallet/
shared/
  domain/  infrastructure/  interfaces/
```

Frontend web (TuEventoFrontend/web/src):

```
App.jsx  main.jsx
assets/  components/  constants/  context/  data/
hooks/  layouts/  pages/  services/  styles/  theme/  utils/
```

Aplicación móvil (TuEventoFrontend/movil/src):

```
assets/  components/  context/  hooks/  navigation/
screens/  services/  theme/  utils/
```

Servicios auxiliares:

```
nsfw-service/
  app.py  requirements.txt  Dockerfile  models/ (se descargan en ejecución)
fake-payment-gateway/
  src/main/java/com/capysoft/fakepaymentgateway/
  pom.xml  Dockerfile  README.md
```

Backend

Perfiles de configuración

El backend usa perfiles de Spring. Se proporcionan los archivos `application-dev.yml` (desarrollo local) y `application-k8s.yml` (Kubernetes), y el perfil activo se selecciona con `SPRING_PROFILES_ACTIVE`. La Tabla 11 resume los parámetros principales de cada perfil.

Tabla 11

Parámetros principales de los perfiles de configuración

Parámetro	Perfil dev	Perfil k8s
Puerto del servidor	8080 (configurable con SERVER_PORT)	8080
Vida del access token	900000 ms (15 minutos)	—
Vida del refresh token	604800000 ms (7 días)	—
Orígenes CORS	http://localhost:3000, http://localhost:5173	Variable CORS_ALLOWED_ORIGINS
Tamaño máximo de archivo / solicitud	10 MB / 12 MB	—
Base de datos	Según SPRING_DATASOURCE_URL	jdbc:postgresql://postgres-service:5432/tuevento
MinIO (endpoint interno)	http://minio:9000	http://minio-service:9000
MinIO (URL pública)	http://localhost:9000 (configurable)	—
Bucket por defecto	tuevento	—
Correo	smtp.gmail.com, puerto 587	—
Redis	Host redis o localhost, puerto 6379	—
Hibernate ddl-auto	—	none
Endpoint de salud	—	Solo health, sin detalles

Nota. Fuente: application-dev.yml y application-k8s.yml. Un guion indica que el parámetro no se define en ese perfil o no fue verificado.

Servicios de Docker Compose

La Tabla 12 lista los servicios que define el archivo `docker-compose.yml`, con sus imágenes, puertos y volúmenes.

Tabla 12

Servicios definidos en docker-compose.yml

Servicio	Imagen o build	Puertos	Volumen	Depende de
postgres	postgres:17-alpine	5434:5432	postgres_data	—
postgres-fake-gateway	postgres:17-alpine	5435:5432	postgres_fake_gateway_data	—
redis	redis:7-alpine	6379:6379	redis_data	—
minio	quay.io/minio/minio:latest	9000:9000, 9001:9001	minio_data	—
minio-init	quay.io/minio/minio:latest	—	—	minio
nsfw-classifier	build: ./nsfw-service	8001:8000	—	—
fake-payment-gateway	build: ./fake-payment-gateway	4001:4001	—	postgres-fake-gateway
backend	build: ./TuEventoBackend/tu-evento	8080:8080	—	postgres, redis, minio, nsfw-classifier

Nota. Fuente: docker-compose.yml.

Variables de entorno

El backend lee su configuración de variables de entorno definidas en el archivo `.env`, que se crea a partir de `.env.example`. La Tabla 13 lista las variables de la plantilla con su descripción. Los valores reales nunca deben incluirse en el repositorio.

Tabla 13

Variables de entorno de la plantilla

Variable	Propósito
`POSTGRES_DB / USER / PASSWORD`	Nombre de la base de datos (tuevento_db), usuario y contraseña
`MAIL_USERNAME / MAIL_PASSWORD`	Cuenta de Gmail y contraseña de aplicación usadas para enviar correo
`JWT_SECRET`	Clave de firma HMAC-SHA256, de al menos 256 bits
`GOOGLE_CLIENT_ID / SECRET`	Credenciales OAuth 2.0 de Google
`VITE_GOOGLE_CLIENT_ID`	Client ID de Google usado por el frontend web
`FACEBOOK_CLIENT_ID / SECRET`	Credenciales OAuth de Facebook
`ADMIN_EMAIL / PASSWORD / ALIAS`	Cuenta inicial del administrador
`MINIO_ROOT_USER / PASSWORD`	Credenciales de administrador de MinIO
`REDIS_HOST / REDIS_PORT`	Conexión a Redis
`SPRING_PROFILES_ACTIVE`	Perfil de Spring activo (dev)
`NSFW_SERVICE_URL / NSFW_THRESHOLD`	Dirección del clasificador NSFW y umbral de decisión
`SIGHTENGINE_API_USER / API_SECRET`	Credenciales de la API de Sightengine
`SIGHTENGINE_MODELS / THRESHOLD`	Modelos de Sightengine verificados y umbral de decisión
`NOTIFICATION_EMAIL_ENABLED`	Habilita o deshabilita las notificaciones por correo
`NOTIFICATION_MAIL_HOST / PORT / USERNAME / PASSWORD`	Configuración SMTP para notificaciones (smtp.gmail.com, 587)

Nota. En la plantilla, NSFW_SERVICE_URL apunta a la dirección de la red de Docker; cuando el backend se ejecuta fuera de Docker debe apuntar al puerto publicado (http://localhost:8001). Las variables que no están en la plantilla se listan en el manual de instalación.

Además de las variables de la plantilla, el código lee otras variables que tienen un valor por defecto y que pueden sobrescribirse (Tabla 14). Los valores por defecto del código pueden diferir de los de la plantilla; por ejemplo, la plantilla usa `NSFW_THRESHOLD=0.6` y `SIGHTENGINE_MODELS=gore`, mientras que el código usa 0.4 y `gore,nudity,weapon` si la variable no se define.

Tabla 14

Variables adicionales leídas por el código

Variable	Valor por defecto	Descripción
SPRING_DATASOURCE_URL	jdbc:postgresql://localhost:5432/${POSTGRES_DB}	URL de conexión a la base de datos
SERVER_PORT	8080	Puerto del backend
APP_OAUTH_GOOGLE_REDIRECT_URI	http://localhost:8080	URI de callback de Google
APP_OAUTH_FACEBOOK_REDIRECT_URI	http://localhost:8080	URI de callback de Facebook
OAUTH_FRONTEND_REDIRECT_URIS	http://localhost:3000/login	URI del frontend tras el flujo OAuth
APP_STORAGE_ENDPOINT	http://localhost:9000	Endpoint de MinIO usado por el backend
MINIO_PUBLIC_URL	http://localhost:9000	URL pública de MinIO
NSFW_SERVICE_URL	http://localhost:8001	URL del clasificador NSFW
NSFW_THRESHOLD	0.4	Umbral del clasificador NSFW
SIGHTENGINE_MODELS	gore,nudity,weapon	Modelos de Sightengine
SIGHTENGINE_THRESHOLD	0.4	Umbral general de Sightengine
SIGHTENGINE_NUDITY_THRESHOLD	0.15	Umbral de desnudez de Sightengine
PAYMENT_GATEWAY	fake	Pasarela de pago usada
FAKE_GATEWAY_URL	http://localhost:4001	URL de la pasarela simulada
WEBHOOK_SECRET	Valor por defecto de desarrollo	Secreto del webhook de pagos
APP_CORS_ALLOWED_ORIGINS	http://localhost:3000, http://localhost:5173	Orígenes CORS permitidos
CORS_ALLOWED_ORIGINS	http://localhost:3000	Orígenes CORS en el perfil k8s
APP_PROFILE_DEFAULT_AVATAR_ID	1	Identificador del avatar por defecto

Nota. Fuente: archivos de configuración del backend.

Autenticación y autorización

El backend autentica a los usuarios con JSON Web Tokens (Jones et al., 2015) firmados con HMAC-SHA256 usando la clave `JWT_SECRET`, y emite refresh tokens para que los clientes puedan renovar sus sesiones. Las contraseñas se almacenan como hashes BCrypt con costo 12. El módulo de seguridad también conserva un historial de contraseñas, de modo que una contraseña nueva pueda compararse con las anteriores cuando el usuario la cambia. El inicio de sesión social con Google y Facebook sigue el marco de autorización OAuth 2.0 (Hardt, 2012).

La autorización se basa en roles. Liquibase (changelog 017) inserta los tres roles del sistema y los estados de usuario listados en la Tabla 15; las migraciones no crean ningún usuario. El administrador inicial se crea al arrancar mediante la clase `DataInitializer` a partir de las variables `ADMIN_*` (Spring, s. f.-b).

Tabla 15

Roles del sistema y estados de usuario

Elemento	Valores	Descripción
Roles	ADMIN	Acceso completo al sistema
	ORGANIZER	Crea y gestiona eventos
	USER	Usuario regular de la plataforma
Estados de usuario	PENDING, ACTIVE, BLOCKED, INACTIVE, DELETED	Ciclo de vida de la cuenta

Nota. Fuente: changelog de Liquibase 017-insert-initial-roles-and-user-status.yaml.

El token de acceso dura 15 minutos (900000 ms) y el refresh token dura 7 días (604800000 ms), según `application-dev.yml`. El refresh token rota en cada uso: al renovar la sesión se emite un par nuevo y el anterior deja de ser válido (`RefreshTokenUseCase`). Para el inicio de sesión, el sistema cuenta los intentos fallidos en una tabla propia (`account_lockout`): tras 5 intentos fallidos dentro de una ventana de 15 minutos la cuenta se bloquea temporalmente (`LoginUseCase`).

Autenticación de WebSocket

Las conexiones en tiempo real se autentican durante el handshake mediante la clase `JwtHandshakeInterceptor`. El cliente envía el token de acceso como parámetro de consulta `token` de la URL `/ws`, porque la API WebSocket del navegador no permite encabezados personalizados. El interceptor valida el token, almacena el identificador del usuario en los atributos de la sesión y rechaza la conexión con una respuesta de no autorizado cuando el token no es válido. La configuración está en `WebSocketConfig.java`: el endpoint es `/ws` (STOMP sobre SockJS), las notificaciones de reserva de asientos se publican en `/topic/events/{eventId}/seats` y las notificaciones personales llegan por `/user/queue/notifications`. Los clientes web y móvil usan `@stomp/stompjs` y `sockjs-client`.

Manejo de errores

Las violaciones de reglas de negocio se lanzan como excepciones que llevan un código de error, y un manejador global de excepciones en el paquete `shared` las convierte en respuestas HTTP consistentes. Por ejemplo, publicar dos comentarios en el mismo evento dentro del intervalo permitido produce el código `COMMENT_RATE_LIMITED`. Este límite de comentarios (diez segundos entre comentarios) es el único límite de frecuencia implementado en el sistema.

API REST y servicios

La API del backend se publica bajo el prefijo `/api/v1`. Esta sección documenta los endpoints de los controladores de los módulos security, event, payment, ticket y wallet, indicando el método HTTP, la ruta, una descripción breve y el rol requerido. Los controladores restantes se listan en la Tabla 22; la documentación interactiva de todos los endpoints está disponible en la interfaz Swagger UI que genera SpringDoc OpenAPI.

Módulo security

Tabla 16

Endpoints de AuthController

Método	Ruta	Descripción	Rol requerido
POST	`/api/v1/auth/register`	Registrar una cuenta nueva	Público
POST	`/api/v1/auth/activate`	Activar la cuenta con el código	Público
POST	`/api/v1/auth/login`	Iniciar sesión con correo y contraseña	Público
POST	`/api/v1/auth/logout`	Cerrar sesión y revocar el token	Público
POST	`/api/v1/auth/refresh`	Renovar el token de acceso	Público
POST	`/api/v1/auth/recover-password`	Solicitar el código de recuperación	Público
POST	`/api/v1/auth/reset-password`	Restablecer la contraseña con el código	Público
GET	`/api/v1/auth/oauth/{provider}`	Redirigir al proveedor OAuth	Público
GET	`/api/v1/auth/oauth/{provider}/callback`	Callback del proveedor OAuth	Público
POST	`/api/v1/auth/google`	Autenticar con un ID Token de Google	Público
POST	`/api/v1/auth/resend-activation`	Reenviar el código de activación	Público
POST	`/api/v1/auth/reactivate-request`	Solicitar la reactivación de la cuenta	Público
POST	`/api/v1/auth/reactivate-confirm`	Confirmar la reactivación	Público
GET	`/api/v1/auth/dev/activate-code/{email}`	Obtener el código de activación (solo desarrollo)	Público

Nota. Fuente: AuthController.java, módulo security.

Tabla 17

Endpoints de UserController

Método	Ruta	Descripción	Rol requerido
POST	`/api/v1/users/change-password`	Cambiar la contraseña	USER
POST	`/api/v1/users/oauth/link`	Vincular una cuenta OAuth	USER
POST	`/api/v1/users/organizer-request`	Solicitar ser organizador	USER
GET	`/api/v1/users/organizer-petition`	Consultar el estado de la solicitud	USER
DELETE	`/api/v1/users/me`	Desactivar la cuenta propia	USER

Nota. Fuente: UserController.java, módulo security.

Tabla 18

Endpoints de AdminController

Método	Ruta	Descripción	Rol requerido
GET	`/api/v1/admin/organizer-requests`	Listar las solicitudes de organizador	ADMIN
PUT	`/api/v1/admin/organizer-requests/{id}/approve`	Aprobar una solicitud	ADMIN
PUT	`/api/v1/admin/organizer-requests/{id}/reject`	Rechazar una solicitud	ADMIN
GET	`/api/v1/admin/organizer-requests/{id}/document`	Obtener la URL del documento adjunto	ADMIN
GET	`/api/v1/admin/events`	Listar todos los eventos	ADMIN
PATCH	`/api/v1/admin/events/{eventId}/status`	Cambiar el estado de un evento	ADMIN

Nota. Fuente: AdminController.java, módulo security.

Módulo event

Tabla 19

Endpoints de EventController

Método	Ruta	Descripción	Rol requerido
POST	`/api/v1/events`	Crear un evento	ORGANIZER
GET	`/api/v1/events/{eventId}`	Obtener un evento por identificador	Público
GET	`/api/v1/events/user/{userId}`	Listar los eventos de un usuario	Público
GET	`/api/v1/events/status/{status}`	Listar eventos por estado	Público
PUT	`/api/v1/events/{eventId}`	Actualizar un evento	ORGANIZER
DELETE	`/api/v1/events/{eventId}`	Eliminar un evento	ORGANIZER
PATCH	`/api/v1/events/{eventId}/status`	Cambiar el estado de un evento	ORGANIZER
GET	`/api/v1/events/public`	Listar eventos públicos	Público
GET	`/api/v1/events/public/city/{cityId}`	Eventos públicos por ciudad	Público
GET	`/api/v1/events/public/category/{categoryId}`	Eventos públicos por categoría	Público
GET	`/api/v1/events/public/date-range`	Eventos públicos por rango de fechas	Público

Nota. Fuente: EventController.java, módulo event.

Tabla 20

Endpoints de calificaciones, respuestas, layout y medios

Método	Ruta	Descripción	Rol requerido
POST	`/api/v1/events/{eventId}/ratings`	Agregar una calificación o comentario	USER
GET	`/api/v1/events/{eventId}/ratings`	Listar los comentarios de un evento	Público
DELETE	`/api/v1/events/{eventId}/ratings/{ratingId}`	Eliminar un comentario propio	USER
PATCH	`/api/v1/events/{eventId}/ratings/{ratingId}`	Editar un comentario propio	USER
POST	`/api/v1/ratings/{ratingId}/replies`	Agregar una respuesta	USER
GET	`/api/v1/ratings/{ratingId}/replies`	Listar las respuestas	Público
GET	`/api/v1/events/{eventId}/layout`	Obtener el layout del evento	Público
PUT	`/api/v1/events/{eventId}/layout`	Crear o actualizar el layout	ORGANIZER
POST	`/api/v1/events/{eventId}/media`	Subir medios del evento	ORGANIZER o ADMIN
GET	`/api/v1/events/{eventId}/media`	Obtener los medios del evento	Público

Nota. Fuente: EventRatingController, EventCommentController, EventLayoutController y EventMediaController.

Módulos payment, ticket y wallet

Tabla 21

Endpoints de pagos, tickets y billetera

Método	Ruta	Descripción	Rol requerido
POST	`/api/v1/payments`	Crear un pago	USER
GET	`/api/v1/payments/{id}`	Obtener un pago por identificador	USER
POST	`/api/v1/payments/{id}/refund`	Solicitar un reembolso	ADMIN u ORGANIZER
POST	`/api/v1/webhooks/payment`	Webhook de la pasarela de pago	Público, firma HMAC-SHA 256
GET	`/api/v1/tickets/{ticketId}`	Obtener un ticket por identificador	USER
GET	`/api/v1/tickets/user/{userId}`	Listar los tickets de un usuario	USER
POST	`/api/v1/tickets/{ticketId}/checkin`	Registrar el check-in de un ticket	ORGANIZER o ADMIN
GET	`/api/v1/wallet/me`	Consultar el saldo propio	USER
GET	`/api/v1/wallet/me/transactions`	Consultar el historial de transacciones	USER
POST	`/api/v1/wallet/credit`	Acreditar saldo	ADMIN
POST	`/api/v1/wallet/adjust`	Ajuste administrativo del saldo	ADMIN

Nota. Fuente: PaymentController, WebhookController, TicketController y WalletController.

Otros controladores

Tabla 22

Controladores adicionales del backend

Módulo	Controladores
ticket	OrderController
category	CategoryController, CategoryEventController
theme	ThemeController
language	LanguageController
notification	NotificationController
profile	ProfileController
seat	SeatController, SeatReservationController, SeatBlockController
section	SectionTypeController, EventSectionController
storage	StorageController
geolocation	GeolocationController

Nota. Fuente: directorios interfaces/rest de cada módulo. Sus endpoints se consultan en Swagger UI.

Base de datos

Motor y acceso

El sistema usa PostgreSQL 17 (PostgreSQL Global Development Group, s. f.). La base de datos principal se llama `tuevento_db` y se publica en el puerto 5434 del host (puerto 5432 del contenedor). La pasarela de pago usa una instancia de PostgreSQL separada en el puerto 5435.

Migraciones del esquema

El esquema se crea y evoluciona con Liquibase (Liquibase, s. f.). El changelog maestro es `db/changelog/db.changelog-master.yaml` y cada cambio es un archivo YAML numerado en `db/changelog/changes/`, por ejemplo `001-create-user-status-table.yaml`, `004-create-user-table.yaml`, `010-create-refresh-token-table.yaml`, 012 (historial de contraseñas), 017 (roles y estados iniciales), `039-create-event.yaml`, `064-create-order-table.yaml` y `074-create-wallet-table.yaml`. Los changelogs son la definición autorizada de cada tabla, columna, restricción e índice.

Resumen del modelo de datos

La Tabla 23 agrupa las entidades principales por dominio. Las definiciones de columnas, restricciones e índices se encuentran en los changelogs de Liquibase.

Tabla 23

Entidades principales por dominio

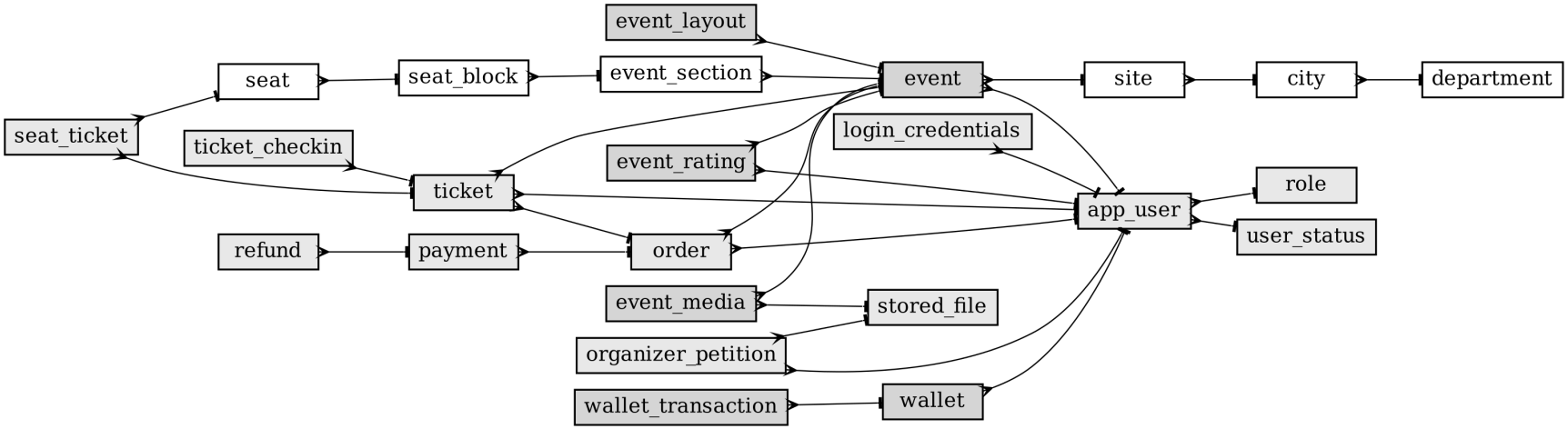
Dominio	Entidades principales
Seguridad	Usuario, estado de usuario, rol, refresh token, historial de contraseñas, activación de cuenta, solicitud de organizador
Eventos	Evento, medios del evento, calificación del evento
Geolocalización y categorías	Departamento, ciudad, sitio, categoría
Asientos	Sección del evento, bloque de asientos, asiento
Tickets	Orden, ticket, check-in del ticket
Pagos y billetera	Pago, reembolso, billetera, transacción de billetera

Nota. Los nombres de las entidades siguen los módulos de dominio. Los nombres exactos de las tablas se detallan en las tablas por dominio de esta sección.

El esquema se define en 90 changelogs numerados del 001 al 090 en `db/changelog/changes/`. La Figura 3 muestra las relaciones entre las tablas principales y las Tablas 24 a 30 detallan, por dominio, las columnas principales, las claves foráneas y las restricciones.

Figura 3

Diagrama entidad-relación de las tablas principales



Nota. Elaboración propia a partir de los changelogs de Liquibase. Cada línea une una tabla con la clave foránea (extremo con pata de gallo, lado de muchos) con la tabla que referencia (extremo con barra, lado de uno). Se muestran las tablas principales; el resto se describe en las tablas por dominio.

Tablas por dominio

Tabla 24

Dominio Security: tablas principales

Tabla	Columnas principales	Claves foráneas	Restricciones
app_user	user_id (PK), user_status_id, role_id, alias, activated	user_status_id a user_status; role_id a role	alias único, no nulo
login_credentials	credential_id (PK), user_id, email, password_hash	user_id a app_user	email único, no nulo
account_activation	activation_id (PK), user_id, activation_code, expires_at	user_id a app_user	activation_code único
account_lockout	account_lockout_id (PK), user_id, failed_attempts, window_started_at, locked_until	user_id a app_user	failed_attempts mayor o igual a 0
organizer_petition	petition_id (PK), user_id, stored_file_id, status	user_id a app_user; stored_file_id a stored_file	user_id único

Nota. Fuente: changelogs de Liquibase.

Tabla 25

Dominio Event: tablas principales

Tabla	Columnas principales	Claves foráneas	Restricciones
event	event_id (PK), user_id, site_id, event_name, start_date, finish_date, status	user_id a app_user; site_id a site	finish_date mayor que start_date; status en DRAFT, PUBLISHED, CANCELLED, COMPLETED
event_layout	layout_id (PK), event_id, layout_data	event_id a event	event_id único
event_media	media_id (PK), event_id, stored_file_id, img_url	event_id a event; stored_file_id a stored_file	No nulos
event_rating	rating_id (PK), event_id, user_id, rating, comment, parent_rating_id	event_id a event; user_id a app_user	rating entre 1 y 5
event_status_log	log_id (PK), event_id, status, reason, changed_by	event_id a event	Eliminación en cascada

Nota. Fuente: changelogs de Liquibase.

Tabla 26

Dominio Geolocation: tablas principales

Tabla	Columnas principales	Claves foráneas	Restricciones
`department`	department_id (PK), name, country_code	—	name no nulo
`city`	city_id (PK), department_id, name, postal_code	department_id a department	name no nulo
`site`	site_id (PK), city_id, name, address, latitude, longitude	city_id a city	name no nulo

Nota. Fuente: changelogs de Liquibase.

Tabla 27

Dominio Seat y Section: tablas principales

Tabla	Columnas principales	Claves foráneas	Restricciones
`event_section`	section_id (PK), event_id, section_type_id, name, price	event_id a event; section_type_id a section_type	price mayor o igual a 0
`seat_block`	block_id (PK), section_id, name, rows, columns	section_id a event_section	rows y columns mayores que 0
`seat`	seat_id (PK), block_id, row_number, seat_number, status, reserved_until	block_id a seat_block	status en AVAILABLE, RESERVED, OCCUPIED, BLOCKED
`seat_log`	log_id (PK), seat_id, previous_status, new_status, user_id	seat_id a seat; user_id a app_user	No nulos

Nota. Fuente: changelogs de Liquibase.

Tabla 28

Dominio Ticket: tablas principales

Tabla	Columnas principales	Claves foráneas	Restricciones
`order`	order_id (PK), user_id, event_id, total_amount, currency, status	user_id a app_user; event_id a event	status en DRAFT, PAYMENT_PENDING, PAID, CANCELLED, REFUNDED, USED
`ticket`	ticket_id (PK), order_id, user_id, event_id, ticket_code, price, status	order_id a order; user_id a app_user; event_id a event	ticket_code único
`seat_ticket`	seat_ticket_id (PK), ticket_id, seat_id	ticket_id a ticket; seat_id a seat	Único (ticket_id, seat_id)
`ticket_checkin`	checkin_id (PK), ticket_id, checkin_time, validated_by	ticket_id a ticket	ticket_id único

Nota. Fuente: changelogs de Liquibase.

Tabla 29

Dominio Payment: tablas principales

Tabla	Columnas principales	Claves foráneas	Restricciones
`payment`	payment_id (PK), order_id, amount, currency, status, wallet_amount	order_id a order	amount y wallet_amount mayores o iguales a 0
`payment_log`	log_id (PK), payment_id, previous_status, new_status	payment_id a payment	No nulos
`refund`	refund_id (PK), payment_id, amount, reason, status	payment_id a payment	amount mayor que 0

Nota. Fuente: changelogs de Liquibase.

Tabla 30

Dominio Wallet: tablas principales

Tabla	Columnas principales	Claves foráneas	Restricciones
`wallet`	wallet_id (PK), user_id, balance, currency, version	user_id a app_user	user_id único; balance mayor o igual a 0; currency en COP
`wallet_transaction`	transaction_id (PK), wallet_id, amount, transaction_type, reference_id	wallet_id a wallet; reference_id a wallet_reference	amount distinto de 0
`wallet_reference`	reference_id (PK), entity_type, entity_id, description	—	entity_type no nulo

Nota. Fuente: changelogs de Liquibase.

Frontend web

El frontend web es una aplicación de página única construida con React y Vite. El estado global se maneja con proveedores de contexto de React para autenticación, tema y notificaciones, y el estado local con hooks de React. El enrutamiento usa React Router, con rutas protegidas para usuarios autenticados, organizadores y administradores.

Todas las llamadas a la API pasan por el cliente HTTP compartido, `httpClient.js`, que adjunta el token de acceso y lo renueva cuando el backend informa que expiró. El contexto de notificaciones abre la conexión WebSocket con el token como parámetro de consulta y entrega los mensajes en tiempo real a la interfaz.

Vale la pena destacar dos componentes especializados. El editor de layouts de eventos (`EventLayoutEditor.jsx`) permite a los organizadores diseñar la distribución de asientos de un recinto sobre un lienzo de Konva, y el layout se almacena como JSON junto con el evento. Los componentes de mapa usan Leaflet para mostrar las ubicaciones de los eventos. El inicio de sesión con Google usa la biblioteca @react-oauth/google con el Client ID definido en `VITE_GOOGLE_CLIENT_ID`. Como Vite incrusta las variables `VITE_*` en tiempo de compilación, la imagen de Docker debe reconstruirse cuando cambian.

Aplicación móvil

La aplicación móvil usa Expo SDK 54 y React Native 0.81, con React Navigation para la navegación de pilas y pestañas. Su dirección del backend se lee de `EXPO_PUBLIC_API_URL` en el archivo `.env` móvil, junto con `EXPO_PUBLIC_GOOGLE_CLIENT_ID`, `EXPO_PUBLIC_FACEBOOK_APP_ID` y `EXPO_PUBLIC_MINIO_HOST`. La dirección de la API debe usar la dirección IP local de la máquina de desarrollo, nunca `localhost`.

El cliente base de la API es `services/api.js`. El servicio de WebSocket (`websocketService.js`) deriva su URL de la misma variable eliminando el sufijo `/api/v1`. La aplicación incluye una pantalla de escáner QR usada para el check-in de tickets.

Integraciones

Inicio de sesión con Google y Facebook

El backend admite el inicio de sesión OAuth 2.0 con Google y Facebook (Hardt, 2012). Una vez que el proveedor confirma la identidad del usuario, el backend crea o vincula una cuenta local y devuelve sus propios tokens. Las credenciales se suministran mediante las variables `GOOGLE_*` y `FACEBOOK_*`.

Notificaciones por correo

Los correos se envían mediante SMTP de Gmail (host `smtp.gmail.com`, puerto 587) usando una contraseña de aplicación en lugar de la contraseña personal de la cuenta. El envío puede deshabilitarse con `NOTIFICATION_EMAIL_ENABLED=false` durante las pruebas.

Almacenamiento de objetos

Los archivos e imágenes subidos se almacenan en MinIO (MinIO, Inc., s. f.) mediante el SDK de AWS S3. La URL pública usada para servir los archivos se define en `MINIO_PUBLIC_URL` y el backend accede al servicio mediante `APP_STORAGE_ENDPOINT`.

Redis

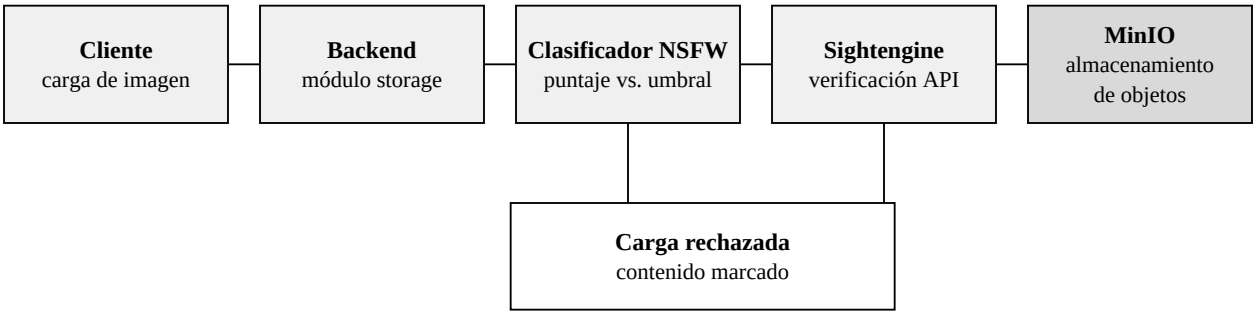
Redis 7 (Redis, s. f.) se ejecuta sin contraseña en el perfil de desarrollo. El backend se conecta mediante `REDIS_HOST` y `REDIS_PORT`.

Cascada de moderación de imágenes

Las imágenes se verifican en dos etapas antes de almacenarse (Figura 4). Primero, el clasificador NSFW local devuelve un puntaje que se compara con `NSFW_THRESHOLD`. Las imágenes que pasan se verifican luego con la API de Sightengine (Sightengine, s. f.) usando los modelos configurados en `SIGHTENGINE_MODELS` y el umbral `SIGHTENGINE_THRESHOLD`. Solo las imágenes que pasan ambas verificaciones se almacenan en MinIO.

Figura 4

Cascada de moderación de imágenes



Nota. Elaboración propia a partir de la configuración de moderación y de la clase SightengineAdapter.

Pasarela de pago

En desarrollo, los pagos los procesa la Fake Payment Gateway, un servicio independiente con su propia base de datos que simula un proveedor real y llama de vuelta al backend mediante un webhook. La pasarela se selecciona con `PAYMENT_GATEWAY` (por defecto `fake`) y su dirección con `FAKE_GATEWAY_URL`. El código define el valor de pasarela `WOMPI`, pero no se implementó ninguna pasarela real. La pasarela simulada no debe usarse en producción.

Patrones de diseño y decisiones técnicas

Patrones arquitectónicos

El backend aplica arquitectura hexagonal: los puertos de entrada definen los casos de uso, los puertos de salida definen lo que la aplicación necesita del exterior y los adaptadores los implementan (Cockburn, 2005). El diseño guiado por el dominio orienta la división en módulos, donde cada módulo posee su modelo de dominio y su persistencia (Evans, 2003). El resultado es un monolito modular cuyos módulos tienen límites claros.

Patrones de diseño

El código usa el patrón Repository (interfaces de dominio implementadas por adaptadores JPA), el patrón Adapter para servicios externos como MinIO, el clasificador NSFW y Sightengine, el patrón Builder mediante Lombok, y componentes mapper generados por MapStruct para convertir entre entidades, modelos de dominio y DTO. La selección de la pasarela de pago mediante configuración sigue el patrón Strategy.

Decisiones técnicas

La autenticación sin estado con JWT mantiene al backend libre de estado de sesión en el servidor. Los refresh tokens permiten usar tokens de acceso de corta duración. La autenticación de WebSocket usa un parámetro de consulta porque los navegadores no pueden enviar encabezados personalizados durante el handshake. La moderación de imágenes se divide en un clasificador local y una API remota para que el contenido obviamente inapropiado se filtre localmente antes de llamar al servicio remoto.

Seguridad

Controles implementados

La Tabla 31 resume los controles de seguridad presentes en el sistema.

Tabla 31

Controles de seguridad implementados

Control	Descripción
Autenticación con tokens	JWT firmado con HMAC-SHA256 y refresh tokens
Protección de contraseñas	Hash BCrypt (costo 12) y verificación de historial al cambiar
Autorización basada en roles	Roles ADMIN, ORGANIZER y USER
Bloqueo de cuenta	Bloqueo temporal tras 5 intentos fallidos en una ventana de 15 minutos
Duración de los tokens	Access token de 15 minutos; refresh token de 7 días con rotación en cada uso
Estados de cuenta	PENDING, ACTIVE, BLOCKED, INACTIVE y DELETED
Validación del handshake WebSocket	Token validado antes de aceptar la conexión
CORS	Orígenes permitidos configurados con APP_CORS_ALLOWED_ORIGINS
Límite de comentarios	Diez segundos entre comentarios del mismo usuario
Moderación de imágenes	Cascada de dos etapas antes del almacenamiento
Gestión de secretos	Secretos en archivos .env excluidos del control de versiones

Endurecimiento pendiente

La configuración por defecto está pensada para desarrollo. La Tabla 32 lista los puntos que deben atenderse antes de exponer el sistema a Internet.

Tabla 32

Endurecimiento requerido antes de producción

Elemento	Estado actual	Acción
Actuator	Todos los endpoints públicos	Exponer solo health e info, o restringir por red
Swagger / OpenAPI	Público	Deshabilitar o restringir
Redis	Sin contraseña	Exigir una contraseña
WEBHOOK_SECRET	Valor por defecto conocido	Definir un secreto privado
Fake Payment Gateway	Solo desarrollo	Configurar una pasarela real
Jenkins y SonarQube	Puertos publicados	No exponer a Internet
Endpoint /api/v1/auth/dev/activate-code/{email}	Público; descrito como solo para desarrollo	Confirmar que solo existe con el perfil dev o eliminarlo antes de producción
Seguridad de transporte	HTTP en configuraciones locales	Usar HTTPS mediante un proxy inverso

Nota. Fuente: notas de seguridad del manual de instalación (Santos Ortiz y Lozano Contreras, 2026a).

Al momento de redactar este documento, el código no contiene un componente de sanitización de HTML, una configuración de Content Security Policy ni un límite de frecuencia global de solicitudes.

Pruebas y calidad

Pruebas existentes

Existen pruebas automatizadas solo para el backend. Son pruebas unitarias ubicadas en `src/test/java/com/capysoft/tuevento/modules/` para los módulos event, security y notification (Tabla 33).No existen archivos de prueba en el frontend web. En total hay 15 archivos Java en `src/test`, de los cuales 14 son clases de prueba (nombres terminados en Test o Tests). El backend incluye H2 como base de datos en memoria para pruebas y el plugin JaCoCo 0.8.12 para medir la cobertura.

Tabla 33

Pruebas unitarias del backend

Módulo	Clases de prueba
event	AddEventRatingServiceTest, AdminChangeEventStatusUseCaseTest, ChangeEventStatusServiceTest, DeleteEventRatingServiceTest, EditEventRatingServiceTest
security	Pruebas de autenticación
notification	Pruebas de notificaciones

Nota. Las pruebas de calificaciones verifican, entre otras reglas, el límite de diez segundos entre comentarios.

Integración continua

El `Jenkinsfile` define las etapas del pipeline mostradas en la Tabla 34. La etapa de pruebas unitarias se ejecuta actualmente con las pruebas omitidas, y la etapa de despliegue solo registra un mensaje de confirmación. SonarQube se ejecuta mediante Docker Compose, pero no forma parte del pipeline.

Tabla 34

Etapas del pipeline de Jenkins

Etapa	Acción
Checkout	Clonar el repositorio
Clean and Compile Backend	`./mvnw clean compile`
Unit Tests	`./mvnw test` (actualmente omitida)
Package Backend JAR	`./mvnw clean package -DskipTests`
Build Backend Docker Image	Construir la imagen del backend
Build Frontend	`npm ci` y `npm run build`
Build Frontend Docker Image	Construir la imagen del frontend
Push to Registry	Subir las imágenes a Docker Hub
Deploy to Kubernetes	Solo registro de confirmación

Manifiestos de Kubernetes

El directorio `k8s/` contiene 8 manifiestos para desplegar el sistema en un clúster (Tabla 35).Las imágenes se publican en Docker Hub (`jhampier23/tu-evento`) desde el pipeline.

Tabla 35

Manifiestos de Kubernetes

Archivo	Propósito
`deployment.yaml`	Deployment del backend Spring Boot
`frontend-deployment.yaml`	Deployment del frontend React
`service.yaml`	Service del backend (ClusterIP)
`frontend-service.yaml`	Service del frontend (ClusterIP)
`ingress.yaml`	Ingress Nginx (api.tuevento.com y app.tuevento.com)
`configmap.yaml`	ConfigMap con variables de configuración
`secrets.yaml`	Secrets con variables sensibles
`hpa.yaml`	Horizontal Pod Autoscaler

Nota. Fuente: directorio k8s/ del repositorio.

Mantenimiento y extensión

Convenciones

Las clases de Java usan PascalCase, los métodos y variables usan camelCase y las constantes usan UPPER_SNAKE_CASE. Los endpoints de la API usan el prefijo `/api/v1` y sustantivos en plural para las colecciones. Los componentes de React usan PascalCase y las utilidades usan camelCase. Los mensajes de commit deben seguir el formato Conventional Commits.

Cómo agregar un módulo al backend

Para agregar un módulo, cree un paquete bajo `modules` con las cuatro capas (`domain`, `application`, `infrastructure` e `interfaces`), siguiendo la estructura de la Figura 2. Defina primero el modelo de dominio y la interfaz del repositorio, luego los casos de uso y los puertos, después la entidad JPA y el adaptador del repositorio, y finalmente el controlador REST. Cada cambio en las tablas debe ser un nuevo changelog de Liquibase numerado agregado al changelog maestro; los changelogs existentes no deben editarse una vez aplicados.

Solución de problemas

Los problemas comunes de instalación y ejecución, como conflictos de puertos, errores de conexión de Liquibase, errores de CORS, fallas de autenticación del correo y problemas de conexión móvil, se documentan con sus soluciones en la sección de solución de problemas del manual de instalación (Santos Ortiz y Lozano Contreras, 2026a).

Limitaciones conocidas

Tabla 36

Limitaciones conocidas de la versión actual

Área	Limitación
Mensajería	Spring Kafka está declarado como dependencia, pero no se implementó ningún productor ni consumidor
Pagos	Solo funciona la pasarela simulada; el valor WOMPI existe sin implementación
Pruebas	Las pruebas del backend cubren tres módulos; el frontend web no tiene pruebas; las pruebas se omiten en el pipeline
Calidad	SonarQube no está integrado en el pipeline de Jenkins
Despliegue	La etapa de despliegue a Kubernetes del pipeline solo registra un mensaje
Seguridad	Sin sanitización de HTML, Content Security Policy ni límite de frecuencia global

Glosario

Tabla 37

Glosario de términos

Término	Definición
DTO	Data Transfer Object; estructura usada para mover datos entre capas
Arquitectura hexagonal	Arquitectura que aísla el dominio de los detalles técnicos mediante puertos y adaptadores
JWT	JSON Web Token; token compacto usado para autenticación
Liquibase	Herramienta que aplica migraciones de base de datos versionadas
NSFW	Not safe for work; contenido no apto para audiencias generales
OAuth 2.0	Marco de autorización para acceso delegado sin compartir contraseñas
RBAC	Control de acceso basado en roles
STOMP	Simple Text Oriented Messaging Protocol; se usa sobre WebSocket
Webhook	Llamada HTTP que un sistema envía a otro para notificar un evento

Referencias

bhky. (s. f.). *opennsfw2* [Software de computadora]. GitHub. https://github.com/bhky/opennsfw2

Cockburn, A. (2005). *Hexagonal architecture*. https://alistair.cockburn.us/hexagonal-architecture/

Docker, Inc. (s. f.). *Docker Compose documentation*. https://docs.docker.com/compose/

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Expo. (s. f.). *Expo documentation*. https://docs.expo.dev/

FastAPI. (s. f.). *FastAPI documentation*. https://fastapi.tiangolo.com/

Hardt, D. (Ed.). (2012). *The OAuth 2.0 authorization framework* (RFC 6749). Internet Engineering Task Force. https://doi.org/10.17487/RFC6749

Jenkins. (s. f.). *Jenkins user documentation*. https://www.jenkins.io/doc/

Jones, M., Bradley, J., y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Konva. (s. f.). *Konva.js documentation*. https://konvajs.org/docs/

Leaflet. (s. f.). *Leaflet documentation*. https://leafletjs.com/reference.html

Liquibase. (s. f.). *Liquibase documentation*. https://docs.liquibase.com/

MinIO, Inc. (s. f.). *MinIO documentation*. https://min.io/docs/minio/linux/index.html

PostgreSQL Global Development Group. (s. f.). *PostgreSQL 17 documentation*. https://www.postgresql.org/docs/17/

React. (s. f.). *React documentation*. https://react.dev/

React Native. (s. f.). *React Native documentation*. https://reactnative.dev/docs/getting-started

Redis. (s. f.). *Redis documentation*. https://redis.io/docs/

Santos Ortiz, J., y Lozano Contreras, C. D. (2026a). *TuEvento: Manual de instalación* (Versión 1.0). CapySoft.

Santos Ortiz, J., y Lozano Contreras, C. D. (2026b). *TuEvento* [Código fuente]. GitHub. https://github.com/lozano2303/TuEvento

Sightengine. (s. f.). *Sightengine documentation*. https://sightengine.com/docs

SonarSource. (s. f.). *SonarQube documentation*. https://docs.sonarsource.com/

Spring. (s. f.-a). *Spring Boot reference documentation*. https://docs.spring.io/spring-boot/

Spring. (s. f.-b). *Spring Security reference*. https://docs.spring.io/spring-security/reference/

The Kubernetes Authors. (s. f.). *Kubernetes documentation*. https://kubernetes.io/docs/home/